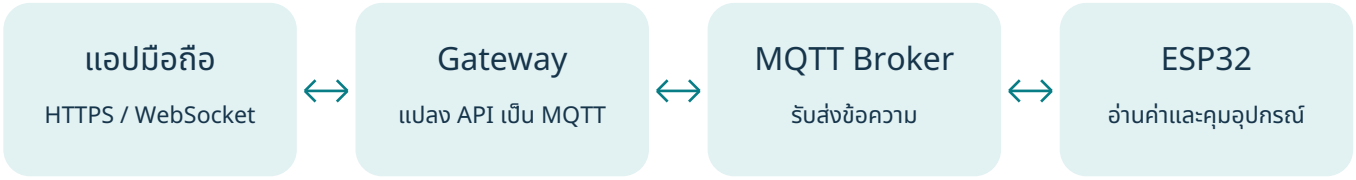
เริ่มจากศูนย์: เชื่อมต่ออย่างไร

อ่านหน้านี้ก่อน | ยังไม่รู้รุ่น ESP32 ก็ทดลองซอฟต์แวร์ได้

แอปรุ่นนี้เชื่อมกับ Gateway บนคอมหรือเซิร์ฟเวอร์ แล้ว Gateway ติดต่อ ESP32 ผ่าน MQTT ไม่ได้จับคู่ ESP32 ด้วย Bluetooth และไม่ใช่การกรอก IP ของบอร์ดลงในช่อง URL โดยตรง



ทำได้ทันที: ช่วง A

เตรียม Gateway จำลองบนคอม แล้วเชื่อมมือถือผ่านสาย USB เพื่อทดลองช่อง URL รหัสเครื่อง การรับค่า และการสั่งงานจำลอง ยังไม่ต้องมีบอร์ดหรือเซนเซอร์ ทำตามหน้า 2-5

หลังเลือกอุปกรณ์: ช่วง B

เตรียมบอร์ด เฟิร์มแวร์ที่รองรับสัญญา MQTT และ Broker แบบ TLS จากนั้นเปลี่ยน Gateway เป็นโหมดฮาร์ดแวร์ ตามหน้า 6-8 เริ่มโดยยังไม่ต่อปั๊มหรือฮีตเตอร์จริง

ไฟล์ที่มีแล้ว	แอป Android, ซอร์ส Flutter, FastAPI Gateway และสัญญาข้อความ ESP32
สิ่งที่ยังไม่มี	เฟิร์มแวร์พร้อมแฟลชสำหรับบอร์ดจริง ผัง GPIO และข้อมูลรุ่นเซนเซอร์/รีเลย์
ผลที่ควรคาดหวัง	ช่วง A ยืนยันซอฟต์แวร์เท่านั้น ยังไม่ยืนยันความแม่นเซนเซอร์หรือการทำงานของอุปกรณ์จริง

เตรียมคอมและโฟลเดอร์

ช่วง A / 1 | ตัวอย่างคำสั่งสำหรับ Windows PowerShell

- ติดตั้ง Python 3.11 ขึ้นไป และ Android SDK Platform Tools จากแหล่งทางการ
- แตก source ZIP จะได้โฟลเดอร์ aqua_hatch ที่มี gateway, lib และ firmware
- ตัวอย่างด้านล่างสมมติแตกไฟล์ไว้ที่ C:\AquaHatch เปลี่ยน path ให้ตรงกับเครื่องจริง

```
cd C:\AquaHatch\aqua_hatch\gateway
python --version
python -m venv .venv
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

ใช้ python.exe ใน .venv โดยตรง จึงไม่ต้องเปิดใช้สคริปต์ activate หรือเปลี่ยน Execution Policy ของ Windows

ตรวจว่าอยู่ถูกโฟลเดอร์

```
Get-ChildItem main.py, requirements.txt
```

ต้องเห็นไฟล์ main.py และ requirements.txt หากไม่พบ ให้กลับไปตรวจตำแหน่งที่แตก ZIP ก่อน

ไฟล์ .env ไม่ได้โหลดเอง

Gateway อ่านค่าจาก environment คู่มือนี้ใช้ $env:... ใน PowerShell โดยตรง การแก้ .env อย่างเดียวจะไม่มีผลหากไม่ได้ตั้งตัวโหลด .env เพิ่มเติม

เปิด Gateway จำลอง

ช่วง A / 2 | รันต่อใน PowerShell หน้าต่างเดิม

สร้างรหัสเข้าถึงใหม่ เก็บไว้เพื่อกรอกในแอป รหัสนี้เป็นของ Gateway ไม่ใช่รหัส Wi-Fi และไม่ใช่บัญชี MQTT

```
$env:DEV_ACCESS_TOKEN = (.\.venv\Scripts\python.exe -c "import secrets; print(secrets.token_urlsafe(32))")
$env:DEVICE_ID = "incubator-01"
$env:SIMULATE_DEVICE = "1"
Write-Output $env:DEV_ACCESS_TOKEN
.\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000
```

คัดลอกรหัสที่พิมพ์ออกมาก่อนข้อความเริ่มเซิร์ฟเวอร์ และเปิดหน้าต่างนี้ค้างไว้ ใช้ Ctrl+C เมื่อต้องการหยุด Gateway

เปิด PowerShell อีกหน้าต่างเพื่อตรวจสุขภาพ

```
Invoke-RestMethod http://127.0.0.1:8000/health
```

status	ต้องเป็น ok
simulated	ต้องเป็น True เพราะยังใช้เครื่องจำลอง
ข้อจำกัดของ /health	บอกว่า API ตอบได้ ไม่ได้ยืนยันว่าต่อ MQTT หรือเซนเซอร์จริงสำเร็จ

หากปิด PowerShell เดิม ค่า $env:... ชุดนี้จะหาย ต้องตั้งใหม่เมื่อเปิด Gateway รอบถัดไป หากใช้รหัสใหม่ ต้องกรอกรหัสใหม่ในแอปด้วย

เชื่อมมือถือผ่าน USB

ช่วง A / 3 | USB ในหน้านี้หมายถึงสายจากมือถือ Android ไปคอม

- ติดตั้ง APK 0.3.1 บนมือถือ แล้วเชื่อมด้วยสาย USB ที่รับส่งข้อมูลได้
- เปิดตัวเลือกนักพัฒนาและ USB debugging บนมือถือ อนุญาตคอมที่กำลังใช้งานเมื่อมือถือถาม
- เปิด PowerShell อีกหน้าต่าง แล้วเรียก adb ตามตำแหน่งที่ติดตั้ง หากมี adb ใน PATH ใช้ adb ได้เลย

```
$adbExe = "$env:LOCALAPPDATA\Android\Sdk\platform-tools\adb.exe"
& $adbExe devices
& $adbExe reverse tcp:8000 tcp:8000
& $adbExe reverse --list
```

คำสั่ง devices ต้องแสดงสถานะ device ถ้าเป็น unauthorized ให้ยืนยันบนมือถือ ถ้าไม่พบ adb.exe ให้ใช้โฟลเดอร์ Platform Tools ที่ติดตั้งจริง

ในแอป: ตั้งค่า > เครื่องและการเชื่อมต่อ > เครื่องจริง

URL เซิร์ฟเวอร์	http://localhost:8000
รหัสเครื่อง	incubator-01
รหัสเข้าถึงเซิร์ฟเวอร์	รหัส DEV_ACCESS_TOKEN ที่สร้างในหน้า 3

กดบันทึกและเชื่อมต่อ ช่องเครื่องจริงใช้เชื่อม Gateway ได้แม้ฝั่ง Gateway กำลังจำลอง จึงยังต้องเห็นข้อความว่าข้อมูลจำลอง เมื่อถอดสายหรือรีบูตมือถืออาจต้องทำ adb reverse ใหม่

ตรวจว่าข้อมูลมาครบ

ช่วง A / 4 | สำเร็จขั้นนี้แล้ว จึงค่อยไปต่อฮาร์ดแวร์

ตรวจ API ด้วยรหัสเดียวกับแอป

ใช้ PowerShell หน้าต่างที่สอง กรอกรหัสจากหน้า 3 อีกครั้ง environment ของคนละหน้าต่างไม่ได้แชร์กันโดยอัตโนมัติ

```
$aquaToken = Read-Host 'Gateway token'
$aquaHeaders = @{ Authorization = "Bearer $aquaToken" }
$aquaUri = "http://127.0.0.1:8000/api/v1/devices/incubator-01"
Invoke-RestMethod -Uri $aquaUri -Headers $aquaHeaders
```

API	มี device_id, config และ telemetry โดย telemetry.simulated เป็น True
หน้าแรกแอป	ตัวเลขอัปเดตและมีป้ายข้อมูลจำลองจากเซิร์ฟเวอร์
ควบคุม	ลองเปลี่ยนโหมดและสั่งงานเฉพาะเครื่องจำลอง ต้องรอข้อความยืนยัน
ห้ามสรุปเกินผล	การจำลองผ่านยังไม่ยืนยัน ESP32 รีเลย์ ปั๊ม ฮีตเตอร์ หรือ AI จริง

มือถือปิดแอปแล้วจะไม่มี Push ในรุ่นนี้ Gateway เก็บประวัติขณะ Gateway ยังทำงาน แต่การปิดโปรแกรม Gateway ทำให้หยุดรับข้อมูลใหม่

เมื่อได้บอร์ด ESP32 แล้ว

ช่วง B / 1 | ตอนนี้ยังไม่ทราบรุ่น จึงยังไม่มีผังสายหรือ GPIO

บอร์ด	ชื่อบอร์ดและชิปบนโมดูล พร้อมรูปด้านหน้า/หลัง เช่น ESP32, ESP32-S3 หรือ ESP32-C3
เซนเซอร์	รุ่นอุณหภูมิ pH flow และระดับน้ำ พร้อมคู่มือและแรงดันสัญญาณ
ภาคกำลัง	รุ่นรีเลย์ Active-high/low แหล่งจ่าย และแรงดัน/กระแสของโหลด
เครือข่าย	Wi-Fi ที่บอร์ดรองรับ ชื่อ MQTT Broker และบัญชีเฉพาะอุปกรณ์

เตรียมเครื่องมือแฟลชตามเอกสาร Espressif

ติดตั้ง Arduino IDE แล้วเพิ่ม Arduino-ESP32 ผ่าน Boards Manager เลือกแพ็กเกจ esp32 by Espressif Systems จากนั้นเลือกชื่อ Board และ COM Port ให้ตรงกับบอร์ดจริง [1]

```
https://espressif.github.io/arduino-esp32/package_esp32_index.json
```

เฟิร์มแวร์ต้องทำอะไรบ้าง

- เชื่อม Wi-Fi ตั้งเวลา UTC ให้ถูกต้อง และเชื่อม MQTT 5 ผ่าน TLS
- ส่ง telemetry/state ทุกประมาณ 2-5 วินาที และรับ command พร้อมส่ง ACK ตามหน้า 8
- อ่านเซนเซอร์จริง คุม Auto และเงื่อนไขป้องกันเอง แม้มือถือหรือ Gateway หลุด
- ตรวจ command_id ซ้ำ วันหมดอายุ และ Manual lease 300 วินาที

โปรเจกต์นี้ยังไม่มี sketch หรือ binary พร้อมแฟลช ขั้นนี้ต้องพัฒนาเฟิร์มแวร์ให้ตรงกับบอร์ดและเซนเซอร์ก่อน ห้ามเดา GPIO หรือต่อโหลดจากตัวอย่างซอฟต์แวร์

เปิด Gateway ฝั่งฮาร์ดแวร์

ช่วง B / 2 | ต้องมี MQTT Broker และเฟิร์มแวร์ที่รองรับแล้ว

แอปยังใช้ URL Gateway เดิมได้ผ่าน USB เริ่มจากหยุด Gateway จำลองด้วย Ctrl+C แล้วตั้งค่าต่อใน PowerShell หน้าต่างเดิม เปลี่ยนค่า .example ให้เป็น Broker จริง

```
$env:SIMULATE_DEVICE = "0"
$env:MQTT_HOST = "mqtt.your-domain.example"
$env:MQTT_PORT = "8883"
$env:MQTT_TLS = "1"
$env:MQTT_USERNAME = "aqua-gateway-01"
$env:MQTT_PASSWORD = Read-Host 'MQTT password'
.\.venv\Scripts\python.exe -m uvicorn main:app --host 127.0.0.1 --port 8000
```

คง DEV_ACCESS_TOKEN และ DEVICE_ID เดิม MQTT_HOST ใช้ชื่อ host เท่านั้น ไม่ใส่ mqtts:// หรือ /path บัญชี Gateway และ ESP32 ควรแยกกันและจำกัด topic ด้วย ACL [2]

Gateway อ่าน	telemetry, state, availability, ack ของ incubator-01
Gateway เขียน	command ของ incubator-01
ESP32 อ่าน	command ของตัวเอง
ESP32 เขียน	telemetry, state, availability, ack ของตัวเอง

ใช้ใบรับรอง Broker ที่ไคลเอนต์เชื่อถือ หากเป็น private CA ให้ตั้ง MQTT_CA_FILE เป็น path ของ CA PEM จริงก่อนเริ่ม Gateway หากใช้ CA สาธารณะให้เว้นตัวแปรนี้โดยไม่ตั้งเป็นสตริงว่าง ไม่ปิดการตรวจใบรับรอง

หลัง ESP32 ส่งข้อมูลปัจจุบัน แอปจึงควรแสดงสถานะเครื่องจริง หาก /health ตอบได้แต่ยังไม่มี telemetry ให้ตรวจ Broker, topic, credential และเวลาบน ESP32 ต่อ

ข้อความที่ ESP32 ต้องส่ง

ช่วง B / 3 | อ้างอิง firmware/PROTOCOL.md ใน source ZIP

telemetry	incubator/v1/incubator-01/telemetry | QoS 0, ไม่ retain
state	incubator/v1/incubator-01/state | QoS 1, retain
availability	incubator/v1/incubator-01/availability | QoS 1, retain
command / ack	.../command และ .../ack | QoS 1, ไม่ retain

ตัวอย่างโครง telemetry ด้านล่างต้องแทน CURRENT_UTC_ISO8601 ด้วยเวลาปัจจุบันที่มี timezone อ่านเซนเซอร์ไม่ได้ให้ส่ง null ไม่ส่ง 0 แทนข้อผิดพลาด

```
{
  "schema_version": 1, "boot_id": "boot-unique-id", "seq": 1,
  "timestamp": "<CURRENT_UTC_ISO8601>",
  "temperature_c": null, "ph": null,
  "ph_calibrated": false, "ph_calibrated_at": null,
  "flow_l_min": null, "water_level_ok": null,
  "device_online": true, "mode": "auto", "manual_until": null,
  "outputs": {
    "pump": false, "heater": false,
    "cooler": false, "tumbler": false
  }
}
```

state มี telemetry และ config ครบ 4 ค่า: temperature_target, ph_minimum, ph_maximum, flow_target ดูตัวอย่างเต็มใน PROTOCOL.md

ACK ต้องมี command_id เดิม และ status เป็น applied, rejected หรือ expired ส่ง applied หลังตรวจเงื่อนไขและตั้งเอาต์พุตสำเร็จ การตอบ PUBACK จาก Broker ยังไม่ใช่การยืนยันจาก ESP32

Last Will ใช้ {"status":"offline"} และส่ง {"status":"online"} เมื่อเชื่อมสำเร็จ พร้อมส่ง telemetry ต่อเนื่องเพื่อให้แอปประเมินความสดของข้อมูลได้

ต่อแบบไร้สายและแก้ปัญหา

ช่วง B / 4 | เก็บเป็นหน้าตรวจเช็กเมื่อเชื่อมต่อไม่สำเร็จ

เมื่อจะถอดสายมือถือออก

ต้องให้มือถือเข้าถึง Gateway ผ่าน HTTPS/WSS ที่มีใบรับรองถูกต้องและเชื่อถือได้ เช่น https://gateway.your-domain.example:8443 แล้วเปลี่ยน URL ในแอป รหัสเครื่องและ Gateway token ยังคงเดิม

แอปปฏิเสธ http://192.168.x.x ตามการตรวจ URL และ Android network policy การอยู่ Wi-Fi เดียวกันจึงยังไม่พอ ต้องตั้ง HTTPS ให้ Gateway ก่อน ไม่ใช้วิธีข้ามคำเตือนใบรับรอง

401	รหัส Gateway ไม่ตรง หรือไม่มี Bearer token
404	รหัสเครื่องไม่ตรงกับ DEVICE_ID
HTTPS ถูกปฏิเสธ	ตรวจ host ในใบรับรอง วันหมดอายุ chain และ URL
URL localhost ไม่ถึง	ตรวจ Gateway ยังรันอยู่ และ adb reverse --list
ขึ้นรอข้อมูล/ปุ่มถูกพัก	ตรวจ telemetry ต่อเนื่อง เวลา UTC และข้อมูลไม่เกิน 15 วินาที
สั่งงานแล้วไม่ยืนยัน	ตรวจ .../command, .../ack และ command_id ให้ตรงกัน แอปรอ ACK ประมาณ 10 วินาที
เปิดรีเลย์แล้ว	แปลว่าเอาต์พุตถูกสั่ง ยังไม่ยืนยันว่าอุปกรณ์ทำงานจริง ต้องตรวจ feedback

ลำดับลงมือและแหล่งอ้างอิง

อ่านต่อเมื่อจบการทดลองซอฟต์แวร์ | ตรวจข้อมูลวันที่ 15 ตุลาคม 2026

- เริ่มที่ช่วง A ให้แอปคุยกับ Gateway จำลองผ่าน USB ได้ก่อน
- ระบุชื่อบอร์ดและรุ่นเซนเซอร์/รีเลย์ แล้วจึงออกแบบผังสายกับเฟิร์มแวร์
- ต่อ MQTT และทดสอบ telemetry/ACK โดยยังไม่ต่อโหลดจริง
- ตรวจเงื่อนไขน้ำต่ำ flow หาย เซนเซอร์เสีย รีบูต และหมดเวลา Manual ตามวงจรจริง
- จึงทดสอบโหลดทีละส่วนและระบบ HTTPS สำหรับใช้มือถือผ่าน Wi-Fi

เอกสารของโปรเจกต์ใน source ZIP

firmware/PROTOCOL.md: รูปข้อความและข้อกำหนดคำสั่ง
firmware/HARDWARE_SETUP.md: ข้อมูลที่ต้องทราบก่อนกำหนด GPIO
gateway/.env.example: ตัวแปรที่ Gateway อ่าน
lib/core/models.dart: เงื่อนไข URL, device_id และข้อมูลสด

เอกสารทางการ

[1] Espressif - ติดตั้ง Arduino-ESP32

https://docs.espressif.com/projects/arduino-esp32/en/latest/installing.html

[2] Eclipse Mosquitto - TLS, authentication และ ACL

https://mosquitto.org/man/mosquitto-conf-5.html

[3] Android Developers - Android Debug Bridge

https://developer.android.com/tools/adb

[4] Uvicorn - ตั้งค่าเซิร์ฟเวอร์

https://www.uvicorn.org/deployment/

คู่มือนี้ตรวจสอบคล้องกับโค้ดแอปและ Gateway มีการทดสอบซอฟต์แวร์จำลอง แต่ยังไม่มี ESP32 เซนเซอร์ และโทรศัพท์เชื่อมต่อให้ทดสอบฮาร์ดแวร์จริง จึงไม่ใช่การรับรองวงจรหรือเฟิร์มแวร์พร้อมใช้งาน